

Bachelor Thesis

Analysis of Failures and Limitations in Generative Retrieval

Richard Takacs

Date of Birth: 24.05.2005

Student ID: 12313036

Subject Area: Generative Retrieval

Studienkennzahl: UJ033560

Supervisor: Svitlana Vakulenko, Adrian Bracher

Date of Submission: 01.04.2026

Department of Information Systems & Operations Management, Vienna University of Economics and Business, Welthandelsplatz 1, 1020 Vienna, Austria

Contents

1	Introduction	1
1.1	Research Questions	2
1.2	Contributions	2
2	Information Retrieval	3
2.1	SEAL Architecture	4
2.2	MINDER Architecture	7
3	Methodology	8
4	RQ1: Failure Mode Classification	11
5	RQ2: Analysis of failures in n-gram-based GR models	22
6	Discussion	34
7	Conclusion	36
	List of Acronyms	

1 Introduction

IR (Information Retrieval) is commonly defined as “the task of identifying and retrieving information system resources that are relevant to an information need.” Traditionally, **IR (Information Retrieval)** has long been structured around the “index-retrieve-then-rank” pipeline [1, 2], with the most popular (and best performing) methods using sparse retrieval, such as BM25 [3] and SPLADE [4]. These rely on lexical representations but suffer when terms do not overlap exactly [5, 6, 7]. A newer paradigm called **DR (Dense Retrieval)**, popularized by frameworks such as DPR [8], utilizes the bidirectional-encoder architectures (typically from BERT [9]) to address this issue by encoding queries and documents into semantic vector representations and then calculating the similarity between query vectors and document vectors, usually via cosine similarity. However, **DR** systems still depend on separate vector index structures [10, 11], and queries and documents are encoded independently with limited semantic interaction [12].

GR (Generative Retrieval) represents an emerging framework in **IR (Information Retrieval)** which aims to directly generate document identifiers through autoregressive language models, such as BART [13] and T5 [14], consolidating the retrieval process into a single end-to-end model [15, 16, 17] by using an **LM** to encode the entire document corpus into the parameters of a single transformer. This architecture offers several advantages: it eliminates the need for separate dense vector indexes, reduces storage needs [18], reduces memory footprint [19], and enables the model to learn document representations jointly with the retrieval objective [20, 21], and potentially enables the optimization of document representations and retrieval objectives while showing comparable performance on evaluation metrics [18, 19, 21]. Despite these promising characteristics, generative retrieval systems exhibit distinct failure modes that differ fundamentally from those observed in traditional retrieval architectures. While **IR** failures typically manifest as low similarity scores in vector space, generative retrieval failures can include hallucination of non-existent document identifiers, incorrect pruning during autoregressive generation, poor performance on dynamically expanding corpora, and challenges with scalability [2, 22, 19, 23], among others. It is also worth noting that many **GR** implementations employ auxiliary data structures for constrained generation, making them not fully end-to-end differentiable.

Existing literature on **GR** documents various challenges and failure cases. However, these failures are typically discussed in isolation with individual papers proposing new methods or architectures. There is little systematic taxonomy that classifies the types of failures that occur in generative retrieval systems and their underlying causes. This thesis addresses this gap

by developing a taxonomy of failures, biases, and limitations for GR systems by synthesizing documented failure cases from the literature and conducting analysis of retrieval outputs from the frameworks SEAL [18] and MINDER models [24] on the NQ (Natural Questions) dataset. The NQ dataset [25] is a benchmark of Google search queries paired with full Wikipedia pages, annotated with the exact section and phrases of the page containing the answer. We also ran MINDER on MSMARCO to improve the generalizability of the results. We refrain from running SEAL on MSMARCO to keep the thesis in scope.

As explained by Li et al. [16], GR processes can be roughly segmented into two components, document retrieval and response generation. In this thesis, we will focus on the failure modes of the former, while briefly discussing the failure modes of the latter as well.

1.1 Research Questions

This thesis addresses two interconnected research questions:

RQ1: What failure modes of GR systems have been identified in the literature, and how can they be organized into a coherent taxonomy?

RQ2: Which failure modes appear in n-gram-based retrieval? What are the conditions in which failures occur?

1.2 Contributions

This thesis makes the following contributions:

1. A synthesis of failure modes documented across generative retrieval research to understand and summarize the issues present in GR and their mitigations.
2. A mapping from abstract failure modes to observable indicators.
3. Quantitative characterization of failure patterns via SEAL and MINDER on NQ, and MINDER on MSMARCO to assess generalizability.
4. Discussion of SOTA challenges, solutions, and future research directions.

The code used for this thesis is available on [Github](#)¹.

¹<https://github.com/richietk/thesis/>

2 Information Retrieval

Generative Retrieval

In this section, we introduce the concept of **GR** (Generative Retrieval) in more depth.

In **GR**, a model is trained to map queries to relevant **docids**. In many cases, seq2seq models like BART [13] and T5 [14] are used, but decoder-only models such as GPT [26] and LLaMa [27] have been explored as well.

A fundamental design decision in **GR** is that of **docids**. A **docid** serves as a unique “index” for each document or passage, which the model typically must autoregressively generate. Usually, each **docid** links to one document, making the mapping bijective [16]. These can be categorized in several ways [16]. Numeric-based **docids** use (series of) numbers to represent documents. Some implementations include unique random integers [28], treating integers as tokenizable text strings [28], hierarchical cluster paths [20], RQ[29]-based **docid** sequences [30]. While atomic IDs require the model to memorize random associations, semantic IDs group similar documents together under shared prefix tokens, improving retrieval quality [23]. Using **LMs** to generate **docids** has also been explored [20, 30].

Text-based identifiers use natural language **docids** which allow the model to use its pre-trained linguistic knowledge rather than learning a new symbolic system from scratch. Among these, the most commonly explored types are document titles [31, 32, 33, 34], n-grams [18, 24, 35, 36], URLs [37, 34], synthetic pseudo-queries [24, 35, 36, 38, 39], and term sets [15, 19].

Title-based **docids** have an inherent advantage in corpora with cleanly annotated unique titles such as NQ [25], but suffer in those without, such as the web-search based MSMARCO [40]. Substring or n-gram identifiers allow any span of text (or group of such spans) within a document to act as its identifier. Pseudoqueries have been shown to prevent forgetting [41] and improve query generalization performance [23]. Term sets have the benefit of being permutation invariant [15]. Please note that this is not always the case [19].

docids are usually static, but in some implementations they can be refined during training, as evidenced by Lee et al. [42], Sun et al. [20].

Hybrid **docids** have also been explored [43].

During the decoding phase, the generation is usually done autoregressively by an **LM** via a beam-search algorithm, which typically generates a ranked list of **docids**. Non-autoregressive systems also exist, however, these are the minority [44, 45]. Decoding is usually constrained to avoid hallucinating nonexistent **docids**. This is usually done via an FM-index [46] (e.g.

in SEAL [18]) or a trie (e.g. in GENRE [31]).

For the query, with encoder-decoder models, the encoder embeds the query into a fixed representation, while the decoder uses that to condition **docid** generation. With decoder-only models, the query tokens are encoded implicitly into hidden states for the decoder.

There has been a number of **GR** implementation approaches, such as combining **GR** with **DR** methods [47], generating a list of n-grams as identifiers [18, 24], using diffusion models to generate **docids** [45, 48], non-autoregressive generation [44, 45], implementing **RLHF** [49], using non-unique **docids** [50], and aiming to integrate the entire **GR** process into a single **LLM** [51]. Multimodal **GR** models have also been explored [52], however, such methods are out-of-scope for this thesis.

2.1 SEAL Architecture

SEAL (Search Engines with Autoregressive LMs (Bevilacqua et al. 2022)) [18], addresses **docid** design challenges by using existing substrings as its retrieval targets. Such challenges specifically addressed are the unavailability of unique metadata (like titles) for all documents, the insufficient granularity of page- and document-level identifiers, as well as the need to force a structure onto the search space, such as hierarchical cluster trees, which can be difficult to construct for large-scale benchmarks. **SEAL** chunks documents into passages and defines identifiers as any n-gram within those passages. A BART model is trained to generate distinctive, query-relevant n-grams that identify documents containing them. **SEAL** constrains generation during decoding so that the model only generates n-grams actually existing in the corpus. For ranking, **SEAL** aggregates information from multiple generated n-grams while downweighing overlaps if an n-gram overlaps with a higher-scoring one already selected. This is implemented so that the final score does not overscore based on repetitive or redundant document content.

Retrieval Mechanism

During the generation phase, **SEAL** enforces that at each decoding step, the language model can only generate tokens that would extend the current partial n-gram into a sequence that exists somewhere in the corpus. As an example, when **SEAL** has generated the partial sequence “earthquakes can be,”, “earthquakes can be predicted” can only be generated if this complete 4-gram exists as a contiguous substring in at least one document [18]. “earthquakes can be predicted” exists in the corpus in the sentence “discussing why the claim that earthquakes can be predicted is false.”

After n-gram generation, in the retrieval phase, for each n-gram, [SEAL](#) identifies every passage containing that n-gram and assigns a relevance score based on the n-gram’s uniqueness and frequency in the corpus. Passages are then ranked by additively aggregating the scores from all n-grams they contain. The FM-index, which [SEAL](#) uses, provides a range of rows in the Burrows-Wheeler Transform matrix [53], and each row corresponds to an occurrence of that n-gram in the corpus. [SEAL](#) then maps these occurrences back to their respective document/passage IDs. This means that a document scores highly not because any single n-gram uniquely identifies it, but because it contains multiple high-scoring n-grams. This provides robustness, since even if some highly ranked n-grams point to an irrelevant document, other n-grams can still guide retrieval toward relevant documents.

This architecture helps explain why [SEAL](#) can overcome vocabulary mismatch issues. While the model cannot generate n-grams that do not exist in the corpus, it can find paraphrases and related terms as long as those appear in corpus documents using BART’s parametric knowledge.

Scoring Mechanism

[SEAL](#)’s failure modes are directly influenced by its scoring function. [SEAL](#) computes n-gram scores by combining the language model’s conditional probability of an n-gram given a query with corpus frequency to favor distinctive n-grams.

The unconditional n-gram probability is computed from corpus statistics:

$$P(n) = \frac{F(n, R)}{\sum_{d \in R} |d|} \quad (1)$$

where $F(n, R)$ is the frequency of n-gram n in corpus R , and $|d|$ is the length of document d .

The n-gram score combines conditional and unconditional probabilities:

$$w(n, q) = \max \left(0, \log \frac{P(n|q)(1 - P(n))}{P(n)(1 - P(n|q))} \right) \quad (2)$$

This formulation promotes n-grams that have high conditional probability given the query ($P(n|q)$), computed by BART, but low unconditional probability in the corpus ($P(n)$). This means that distinctive, query-relevant n-grams are prioritized.

It is important to highlight that $P(n|q)$ is computed autoregressively by BART and depends on both the n-gram n and the query q . Consequently, the same n-gram string can receive different scores for different queries. Therefore, n-gram scores are query-dependent.

Finally, the document score aggregates contributions from multiple non-overlapping n-grams:

$$W(d, q) = \sum_{n \in K^{(d)}} w(n, q)^\alpha \cdot \text{cover}(n, K^{(d)}) \quad (3)$$

where $K^{(d)}$ is the subset of generated n-grams n matching document d . An n-gram is included in $K^{(d)}$ only if it has at least one occurrence in d that does not positionally overlap with a higher-scoring n-gram’s occurrence. The parameter α is a hyperparameter. The coverage weight $\text{cover}(n, K^{(d)})$ downweights individual n-grams whose tokens overlap with tokens of higher-scoring n-grams in the same document, preventing passages from receiving inflated scores when they contain many lexically similar n-grams.[18].

Here, it is worth noting that **SEAL**, generates fixed token-length ngrams (by default $k = 10$, except for titles). However, they save all partially-decoded sequences from the beam-search [18]. Because the system records this history, the final score of a passage is always the sum of strings of different lengths. As expected, n-gram length is strongly correlated with frequency ($\rho = -0.835, p < 0.001$).

To illustrate **SEAL**’s n-gram based scoring, let’s examine the query “who sings does he love me with reba”. The top-ranked passage (titled “Linda Davis”) matched the following keys (excerpt):

N-gram	Corpus Freq. $F(n, R)$	Score $w(n, q)$
</s> Linda Davis @@	9	216.33
"Does He Love	23	196.06
Reba	1,851	103.56
country singles	777	83.47
country music singer	4,024	39.20

Table 1: Example n-gram keys for query “who sings does he love me with reba”. Lower corpus frequency correlates with higher scores for query-relevant n-grams.

It is also worth noting that while the additive nature of the scoring mechanism could theoretically bias retrieval toward longer documents in a corpus of varying document lengths, this effect is neutralized by dividing $F(n, R)$ by the total number of tokens in the corpus R .

2.2 MINDER Architecture

The **MINDER** (Multiview Identifiers Enhanced Generative Retrieval (Li et al. 2024)) framework [24] extends the n-gram based approach of **SEAL** by introducing synthetic and multiview identifiers. **MINDER** is motivated by the observation that single-view identifiers, such as document titles or extractive n-grams, often lack the contextualized information necessary to satisfy complex queries, especially if those require rephrasing.

To address this, **MINDER** assigns three distinct “views” of identifiers to each passage:

1. Title of the document
2. N-grams, similar to **SEAL**
3. Pseudo-queries, i.e. synthetic identifiers generated via a query-generation model that reflect potential queries

The inclusion of pseudo-queries is intended to bridge the gap between user queries and document content. While **SEAL** is limited to the exact word orderings present in the corpus, **MINDER** thus allows the model to match queries against rephrased or summarized versions of the document via pseudo-queries.

MINDER relies on the conditional probability $P(i|q)$ of the identifier i given query q , computed by the autoregressive language model. Because pseudo-queries are typically much longer than n-grams, their raw log-probabilities are naturally lower due to the multiplicative nature of autoregressive decoding. To ensure that pseudo-queries can compete with shorter identifiers, **MINDER** introduces a biased score to offset this length penalty. The final relevance score for a passage p is an aggregation of the scores from all predicted identifiers that appear in that passage across all three views:

$$S(q, p) = \sum_{i \in I_p} s(i, q) \quad (4)$$

where I_p is the set of identifiers generated by the model for passage p , and $s(i, q)$ is the language model score for identifier i .

To manage multiple identifier types, **MINDER** uses identifier-specific prefix tokens (e.g., <TS> for titles, <QS> for queries). During inference, the model is prompted with an identifier prefix and the query, and constrains the generation to valid sequences within the respective view.

Evolution of SEAL/MINDER

While this thesis focuses primarily on the foundational [SEAL](#) and [MINDER](#) architectures, it is essential to mention their successors like [LTRGR](#) and [DGR](#).

All four systems share a common BART-based autoregressive language model paired with an FM-Index for constrained decoding. [MINDER](#) builds upon [SEAL](#) by adding “multiview identifiers”, such as pseudoqueries. [LTRGR](#) introduced a second training phase using a supervised Rank Loss, to minimize the score of negative passages relative to positive ones. [DGR](#) uses a cross-encoder model as a re-ranker to improve the ranking list.

This thesis intentionally focuses its failure analysis on the core of such systems, as [LTRGR](#) and [DGR](#) function as “optimization layers” built upon [MINDER](#). Therefore, many of the fundamental behavioral biases are inherited from the underlying generative mechanics. In case that does not hold, we will specifically mention the other systems.

3 Methodology

Our methodology is twofold. Firstly, we will analyze the failure modes identified qualitatively, drawing from an extensive list of related works, as several papers have already identified failure modes in isolation related to their respective [GR](#) model contributions. Secondly, we ran [MINDER](#) on the [NQ](#) (Natural Questions) [25] and [MSMARCO](#) (Microsoft Machine Reading Comprehension (Bajaj et al. 2018)) [40], to analyze their outputs. This helps us to empirically connect our qualitatively analyzed failure modes to observable failure indicators in n-gram-based retrieval. We choose [MINDER](#), as it serves as a foundational model in the n-gram based [GR](#) space.

We chose [NQ](#) (Natural Questions) as it is an academic standard for retrieval benchmarking. The [NQ](#) retrieval corpus consists of a Wikipedia dump split into approximately 21 million passages of 100 tokens each. [MSMARCO](#) is also a standard evaluation dataset, which contrasts with [NQ](#) in that its metadata is messier, inconsistent, or even incomplete, which might better reflect real-world use cases than the clean Wikipedia dump of [NQ](#).

The [SEAL](#) and [MINDER](#) output dataset contains the results of 6,515 queries from the [Natural Questions](#) benchmark. Additionally, [MINDER](#) was run on the [MSMARCO](#) passage dev set across 6,980 queries to evaluate generalizability to a noisier, web-search-based corpus. For each query, the following relevant information is available:

- The original natural language query

- GT answer string(s)
- Annotated GT passages containing the answer
- Negative and hard negative passages
- The model’s top-100 retrieved passages, ranked by passage score, each containing:
 - Passage title and text
 - Aggregate document score
 - The set of matched identifiers and their respective scores

From the output data available, the n-gram keys are deemed most useful to understand SEAL’s and MINDER’s retrieval decisions. Each key entry contains three parts:

1. The n-gram string (e.g., “ Reba”, “</s> Linda Davis @@”)
2. The corpus frequency of how many times this n-gram appears across all documents
3. The n-gram score computed via the scoring function

The MINDER output is structurally identical to that of SEAL’s, including being run on the same 6,515 queries. The main difference arises from the pseudoqueries, which are appended to the *text*. For MSMARCO, MINDER output follows the same structure across 6,980 queries.

On NQ, the output dataset (of both systems) has an average per-query annotated correct passage count of 8.46, with the minimum being one and the maximum being 101. This is important to analyze to accurately identify evaluation metrics. To illustrate the distribution, please consult Figure 1.

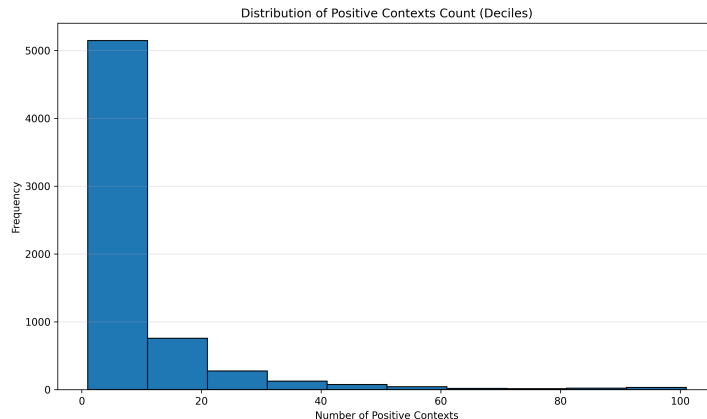


Figure 1: Distribution of annotated correct passage counts per query on the NQ dataset.

For MINDER on MSMARCO, the average annotated correct passage count per query is much lower, with the average having around 1.32 positive passages. The distribution is comparable to that of NQ.

Evaluation Metrics

We used Hits@ k to determine retrieval performance for [SEAL](#) and [MINDER](#). For most of our evaluations, we use $k = 1$, $k = 10$, and $k = 100$, following previous works. To analyze distribution, we bin the results into deciles and report them. Additionally, when relevant, we report the Spearman rank correlation values to analyze the relationship between variables. We also include MRR, nDCG, and Recall when appropriate.

4 RQ1: Failure Mode Classification

We categorize the failure modes by which part of the GR process it occurs in. We create four such categories:

- Representation
- Training
- Inference
- Response

Please note that the last category “response”, which refers to the response generation via an LLM post-retrieval, is not traditionally considered part of GR and is considered part of RAG. Nevertheless, we will identify failure modes in that category as well to achieve our goal of creating a comprehensive taxonomy.

Representation

DocIDs not being sufficiently distinct, conflating similar documents GR systems also suffer from cases where the mapping from document content to identifiers is not sufficiently distinct [54]. This is especially prominent with large corpus sizes. Additionally, when using synthetic queries [55] or semantic clustering [28] as identifiers, multiple documents may share quasi-identical or highly similar representations, causing the model to conflate distinct documents [56, 20, 16]. This is frequently observed when using semantic clustering as identifiers, where multiple documents with polysemous terms, such as "Apple" referring to both the company and the fruit, are forced into the same cluster [28]. Similarly, when identifiers are not unique across distinct meanings, such as an "inverted file index" (IVF) and "in vitro fertilization" (IVF) potentially sharing identifiers in certain configurations, the model could conflate unrelated documents, leading to retrieval precision issues.

Non-unique identifiers In frameworks relying on substring-based identifiers, a single generated identifier (e.g., "The apple") may exist in multiple distinct passages across the corpus. This creates a one-to-many mapping that prevents the model from uniquely resolving a specific document, often necessitating complex post-hoc scoring or re-ranking mechanisms to disambiguate the results [18, 24]. This ambiguity forces the GR system to rely on auxiliary scoring statistics to filter the retrieved candidates.

Specific terms inaccurately split into tokens If an important identifier (e.g. a proper noun) is split into many meaningless subwords, the model might struggle to generate it consistently compared to common words. This can happen even though BART uses BPE, which allows it to handle “out-of-vocabulary” words by breaking them down into known subunits. Accordingly, if BART doesn’t have good representations for domain-specific jargon, e.g. for rare medical terms, retrieval can fail [57]. As an example, rare medical drug names like “Zyloprim,” are split into meaningless subword units (e.g., "Zy-lo-prim"), and the LM can struggle to generate these identifiers consistently. Because GR relies on the model’s ability to decode the correct sequence of tokens, these, separately meaningless, tokens can cause the model to assign low probabilities to valid identifiers. This issue is particularly pronounced in non-English or non-Latin scripts, where a single word may be represented by a significantly larger number of tokens. Non-English, primarily non-Latin, texts use several times more tokens per text. As an example, “Hello World” in Hindi uses 23 tokens using the tokenizer of BART-Large, which many of the models use. In English, “Hello World” is two tokens long. This also has the potential to impact the scalability and memory requirements of GR models when ported to foreign languages.

DocID assignment is often a static pre-processing step decoupled from the E2E gradient Many GR implementations use static, pre-computed document identifiers, which act as a decoupled preprocessing step in the retrieval pipeline [50]. Because these DocIDs are typically assigned before the model training begins,

Non-differentiability can be considered a failure mode in the optimization of GR systems. Since in most models docids are assigned via a non-differentiable argmax function (which necessitates a static preprocessing phase), such models form a “fake end-to-end” framework, according to Yang et al. [50]. Static, pre-computed document identifiers are a decoupled preprocessing step in the retrieval pipeline, as DocIDs are typically assigned before the model training begins, thus the indexing module cannot receive gradient updates from the retrieval decoder. Consequently, the model is stuck with a potentially sub-optimal document representation that it cannot refine, forcing the decoder to “meaninglessly mimic” the index rather than autonomously learning to shape the identifier space to maximize retrieval accuracy [50]. This restricts the model’s ability to move beyond rote memorization. Yang et al. [50] proposes the use of the Straight-Through Estimator [58] as a reparameterization mechanism.

Non-discrete docids (such as embedding vectors) have been explored as

a solution by replacing the argmax operation with a continuous similarity function, essentially forming a hybrid between traditional GR and DR models [59, 43, 47].

Over-reliance on metadata; performance drops when metadata is missing/messy GR models frequently exhibit an over-reliance on document metadata, such as titles, which serve as highly discriminative "shortcuts" during training [18, 24]. In datasets like NQ (Natural Questions), where metadata is clean and unique, this manifests as high performance, but the model fails to learn the actual content of the passages. If a document lacks a descriptive title or contains noisy metadata, the model’s performance degrades sharply, making it poorly suited for datasets like MSMARCO (Microsoft Machine Reading Comprehension (Bajaj et al. 2018)) where metadata is often inconsistent [40].

Lexical mismatch between user queries and the generated identifiers Even when the model possesses the parametric knowledge to identify a relevant document, it may fail to bridge the lexical gap between a user query and the document’s identifiers [60]. For instance, a query asking about “heart attack” may fail to trigger the docID “myocardial infarction” if the model has not explicitly learned that association during the identifier generation phase. The model’s parametric knowledge partially mitigates this, however, it is still prevalent in substring-based retrieval, where the model is constrained by the exact word orderings present in the corpus. [18].

In n-gram-based docids, n-grams must be continuous

As discussed previously, a fundamental structural constraint of autoregressive, beam-search, and n-gram-based retrieval is that such n-grams must be strictly contiguous substrings within the corpus. This means the model cannot match n-grams that would semantically connect a query to a passage if the relevant terms are separated by intervening words or are cut off between passages.

Consider a passage stating “Tom likes pizza and tomatoes” and a query “does Tom like tomatoes?” The relevant n-gram “likes tomatoes” does not exist as a contiguous substring in the passage due to the intervening words “pizza” and “and.” This issue is partially mitigated by the fact that n-grams “and tomatoes,” “pizza and tomatoes,” “tomatoes.” are still generated and scored, however, this is not a robust solution. Introducing skip-grams, using methods such as permutation-invariant decoding [15], or forcing overlap between passages could theoretically be solutions to this problem.

Training

Sequential indexing of new documents causes the model to "forget" the initial corpus Continuously updating the corpora has been a central challenge in GR. When a model is fine-tuned to index new documents, it often suffers from forgetting, where the ability to retrieve previously learned documents degrades. Several models have been proposed for solving such issues, such as using frozen pre-trained LMs [61, 62] and updating only specific PQ clusters in PQ-based models [41].

In environments where corpora change, GR models exhibit several types of failure modes [41, 63, 47, 64]. Forgetting occurs when updating a model with new documents degrades its ability to retrieve previously indexed documents. Experiments demonstrate that sequential indexing of new batches can cause indexing accuracy for the initial corpus to drop by more than 25 points, necessitating retraining and increasing costs [64]. The opposing failure is an excessive bias against recency, where models fail to retrieve newly added content, favoring documents from the original training corpus [63].

Experiments indicate that sequential indexing can cause accuracy for the initial corpus to drop by over 25 points, requiring costly retraining cycles [64]. While generative models capture coarse-grained semantic clusters effectively, they struggle to accurately memorize and decode fine-grained document distinctions [65]. According to Yuan et al. [47], error rates (defined as the % of tokens predicted at a position that do not match the GT token in a docid) for NCI [38] increase significantly at later token positions in docid sequences, rising from approximately 1% at the first position to over 12% by the sixth position. While nonstandard, diffusion-based models also experience forgetting, as evidenced by Zhao et al. [45]. This phenomenon is a foundational instability in GR models where the parametric memory is susceptible to interference from new inputs, effectively "overwriting" old information or previously stored factual mappings when the system attempts to integrate new knowledge [63].

GR models also exhibit forgetting during initial training as well. Analysis of training dynamics shows models frequently "forget" and "re-learn" document-to-identifier mappings, with approximately 88% of documents undergoing at least one forgetting event during training [64].

An additional issue identified is the problem that newly added documents are practically inaccessible until model weights are updated and the model retrained, unlike dense retrieval systems, which can index new embeddings instantly without retraining the encoder [17].

Deleting a document from the corpus does not automatically update model parameters Unlike traditional IR (Information Retrieval) systems where a document can be removed from an inverted index, deletion in GR is non-trivial because the document identity is "baked" into the model's weights [17]. If a document is removed for accuracy or privacy reasons, the model's parameters may still "know" the docID and continue to attempt to generate it at inference time. Without a full retraining cycle, it remains challenging to purge specific knowledge from the LM index, which is a significant challenge for production-ready compliance and content management systems [64].

Implicit biases learned from the training data GR models often exhibit biases learned from the training data. A prominent example is Western-centricity, as the models are trained on datasets which are overwhelmingly English-based and Western-focused, such as the English Wikipedia [25, 40]. English content in public corpora is itself not culturally neutral, as a substantial portion reflects Western (particularly US and Eurocentric) perspectives and framing simply because such sources dominate English public corpora. As BART (which SEAL and MINDER use) was trained on the English Wikipedia and on BookCorpus [66, 67], it can be presumed that BART would implicitly default to Western interpretations in ambiguous cases. Other commonly used language models, such as T5 and LLaMa-3 [14, 27] were also trained on overwhelmingly English data. Similar bias was also demonstrated for LLMs by multiple papers [68, 69, 70].

Similarly, the NQ dataset is also based on the English Wikipedia [25], adding to the issue. Such training data biases, including Western-centricity, can also be shown in other commonly used datasets, such as MSMarco [40], Trivia QA [71], and the KILT datasets [72].

For example, a query regarding "the capital" may default to a U.S. context, ignoring historical or non-Western alternatives, simply because the underlying LM has stronger parametric associations with Western-centric data. This cultural bias is integrated into the model's generation process, which might reinforce cultural stereotypes and Eurocentric worldviews [73].

Performance drop when applied to unseen query distributions or datasets If the model is trained through too few epochs, it can underfit, and retrieval can be noisy. If it is trained on too many, the model memorizes the training set too well and loses the ability to generalize to new queries. This is especially problematic considering that GR models are prone to rote memorization of docids and are not robust to misspellings and changes in

word order, according to Liu et al. [60]. A model trained on NQ (Natural Questions) performs well on general questions but may struggle when faced with specialized legal or technical documentation because its identifier generation strategy is overfitted to the phrasing and vocabulary of the training data.

Related is the challenge of “zero-shot” retrieval. Supervised GR models usually perform significantly worse on datasets they weren’t explicitly trained on [60, 63]. They usually lack robust matching capabilities like that of BM25 or Contriever [74] in zero-shot settings. According to Pradeep et al. [23], higher Jaccard similarity between pseudoquery docids and evaluation queries correlates with higher performance, indicating that models struggle when the query distribution is out of distribution. He also notes that models can overfit on the type of phrasing in the synthetic queries.

Similarly, according to Liu et al. [60], NCI [38] performance significantly drops (from 71.2 to 27.7 R@100) when being evaluated on unseen documents. Using N-gram based docids is potentially a solution, as they tend to perform better than numeric ones [18].

If knowledge distillation via a cross-encoder model is used, that “teacher” can be wrong, harming the GR model Knowledge distillation is sometimes used to improve GR performance, but it introduces the risk of propagating “teacher” errors [36]. If a cross-encoder incorrectly ranks a document as highly relevant, the GR model learns to force a high probability on that incorrect docID. This creates a distortion in the identifier space where the student model adopts the teacher’s incorrect rankings and bakes the teacher’s biases into the GR system [35].

Not all documents have labeled queries in datasets Many documents in a large corpus lack associated labeled queries, meaning the GR model has no direct supervision for those DocIDs during training [17]. While training on synthetic queries can mitigate this, these pseudo-queries often lack the nuance of real human information needs, causing a mismatch between the training and inference distributions. As a result, the model may struggle to learn how to map meaningful questions to these “unlabeled” docIDs, resulting in poor retrieval performance for a large portion of the corpus that is technically indexed but not effectively queryable [23].

The model’s capacity fails to scale with corpus size Generative retrieval models face significant challenges regarding parametric capacity, as models have increasingly been tested on large corpora. The “parameter

budget” of the transformer can become insufficient to encode nuances of a massive corpus. According to Pradeep et al. [23], the retrieval effectiveness of the DSI [28] model drops significantly on the full MS MARCO dataset [40] (8.8M passages) as the corpus scales, with MRR@10 falling from 80.3 on a 100k subset to just 24.2 on the full collection. They found that a T5-XXL (11B) [14] model underperformed a T5-XL (3B) model under identical settings. This is in accordance with Bevilacqua et al. [18] who argued that memorizing fixed document identifiers fails to scale when applied to the 21M passages of the NQ [25] Wikipedia corpus. Yuan et al. [47] reached a similar conclusion, stating that when the corpus size exceeds the available memory of the parameters, it hinders retrieval accuracy. They showed that scaling the NCI [38] model from a 334k to a 1M document corpus on the NQ [25] dataset resulted in an 11.0 percentage point drop.

As explained by Zhou et al. [34], the use of unstructured atomic identifiers (i.e. assigning unique, non-semantic integers as `docids`) does not fix this issue either. Since the output projection layer has dimensions of hidden size $\times$ vocabulary size [75, 28], it makes this approach prohibitively expensive for large corpora. As an example, 8.8 million documents (the amount of passages in MsMarcoFull [40]) with a hidden size of 768 (the hidden dimension of the T5-base model [14]) adds approximately 6.8 billion parameters to the output layer alone, making this approach prohibitively expensive for large corpora. Sequential identifiers reuse a fixed vocabulary across multiple decoding steps, keeping parameter count constant regardless of corpus size [17, 23]. As an example, PQ-based `docids` allows for `docid` tokens to be “sharable” among documents.

Inference

Beam search prematurely discards relevant docIDs based on low initial token probabilities Prefix pruning, also referred to as false pruning occurs during autoregressive generation of `docids` when the beam search prematurely discards the prefix of a relevant docid because its cumulative probability at a specific step is lower than that of competing candidates [15, 19, 20, 30]. This occurs as beam search tends to get stuck in the local optima [76], as it perceives only preceding tokens and lacks visibility into subsequent ones, and thus, once a prefix is pruned, the system cannot recover the correct document regardless of subsequent token probabilities. There have been efforts to combat this, such as implementing backtracking [77] and replacing beam-search with a set-based sampling [15]. Non-autoregressive generation does not exhibit this issue [44, 45]. Empirical analysis by Zeng et al. [19] indicates that even with large beam sizes, constrained beam search

often fails to match brute-force decoding performance, suggesting that relevant [docids](#) are frequently eliminated during search.

The autoregressive nature of decoding inherently favors shorter n-grams over longer ones [47], drowning out highly specific and discriminative n-grams. Additionally, longer n-grams naturally introduce more chances of false pruning. These indicate that the global optimum itself could be a short or irrelevant prefix, making the model unable to accurately pinpoint correct passages by preferring the shortest valid [docid](#) path or the one with the most frequent initial tokens, regardless of their actual relevance to the query [30].

High compute cost of autoregressive generation compared to dense/sparse retrieval [GR](#) models incur significantly higher inference latency due to the autoregressive nature of generating identifier sequences token-by-token [47]. While sparse or dense retrieval involves efficient index lookups, [GR](#) requires the model to “think” and generate tokens, a process that is FLOP-intensive and scales poorly with beam size and sequence length. This makes it difficult to deploy [GR](#) in real-time search applications where response time is critical.

There has been some variance in the severity of this issue. Zhou et al. [34] found that Ultron-PQ had half the latency of [DPR](#) for the [MSMARCO](#) 320k [40] corpus. On the other hand, Yuan et al. [47] states that their analyzed [NCI](#) [38] model had a lower efficiency due to the autoregressive [docid](#) generation than the [AR2](#) dense retriever [78] for the [NQ](#) 334K dataset.

Latency also depends on [docid](#) type, corpus size, and beam-size. Atomic identifiers are much more FLOP efficient than sequential identifiers [23]. Constraining the decoding also comes with overhead, proportional to beam-size [18].

Replacing transformer-based architectures with alternatives like [RetNet](#) [79] or [Mamba](#) [80] could be a potential solution to latency (and to an extent scalability), as explained by Li et al. [16]. These are, however, largely unexplored yet, and are out-of-scope for this thesis.

Preference for shorter docIDs, as each extra token reduces the total probability of the sequence [GR](#) models often exhibit a length bias, where they disproportionately prefer shorter identifiers because token probabilities are multiplicative and tend to shrink as the sequence length increases [47, 18, 24, 15]. Consequently, the model may favor a shorter, more generic DocID, e.g. “War”, over a longer, more discriminative one, e.g. “The Napoleonic Wars”, because the shorter sequence is “safer” to generate without making a mistake. This bias drowns out highly specific, unique identifiers,

essentially punishing long but precise DocIDs [30].

Error rates increase at later token positions in a docID sequence, fine-grained features disappear Empirical analysis of GR generation reveals that models frequently experience "drift" at the end of long identifiers, where fine-grained features of the DocID are lost as the model "loses its train of thought" [47]. Error rates for generated tokens often increase from 1% at the first position to over 12% by the sixth position. This suggests that the GR framework struggles to maintain the structural integrity of complex DocIDs, leading to invalid or near-miss identifiers that point to the wrong document or result in a total failure to retrieve any target at all [65].

Passages retrieved are not sufficiently distinct Generative retrieval may retrieve a list of passages that are topically redundant, failing to provide the diverse perspectives needed for ambiguous queries, such as "What is IVF?" [56, 20]. The model's guess might match only one intent (e.g., the medical procedure), completely ignoring others (e.g., the inverted file index). This lack of diversity in the ranked list is a critical failure mode in GR, as the models often lack a mechanism to force the generation of distinct DocIDs that cover different facets of an ambiguous query, unlike traditional IR systems that can employ diversity re-ranking [16].

Generation of docIDs that do not exist in the corpus Related is the problem of hallucination, in which the model generates invalid (with respect to the pre-defined document index) identifiers. Constrained decoding is often used to eliminate this problem with solutions such as constraining the docid generation through an FM-index [18] or a trie [31]. Using Bloom filters [81] or cuckoo filters [82] could be another method. There have been several implementations of constrained decoding, such as SEAL [18] and MINDER [24].

Several GR solutions use intentional hallucination, such as MINDER[24], where the generation of additional identifiers, such as pseudoqueries, is achieved through intentional, constrained hallucination of docid generation.

Response

The LLM summarizer generating what is not present in the retrieved text In the response generation stage, the LLM may hallucinate information not contained within the retrieved documents, relying instead on its pre-trained parametric knowledge [83]. This is usually tackled by methods

such as self-reflection [84, 85] or by incorporating a non-parametric memory component, such as DPR, to condition generation on retrieved documents at runtime [86], the latter commonly known as RAG.

Context window being limited Generative systems often struggle with the context window bottleneck when synthesizing answers from a large set of retrieved documents [16]. If a user query requires information from 20 disparate sources, but the LLM’s context window is limited, the model will truncate the input or ignore relevant information at the end of the context. This leads to incomplete answers that fail to synthesize the retrieved evidence, forcing the model to operate on a limited subsection of the retrieved data [87].

“Lost in the Middle” phenomenon LLMs demonstrate a specific tendency to ignore documents or information placed in the middle of a retrieved context window, while heavily over-weighing information at the beginning or end of the sequence [88]. In a GR setup, if the system retrieves the most relevant passage for the "middle" of the context, the LLM is statistically likely to ignore it. This means the system’s performance is sensitive to the order in which documents are presented to the generator, regardless of their actual relevance to the query.

The model outputting sensitive memorized training data Privacy surrounding LLM-memorized content and the risk of models producing unsafe content is a central point [16]. Ensuring generative systems don’t violate user privacy or inadvertently return malicious text remains an open challenge. This encompasses malicious code, stereotypes, bigotry, and PII information, among others.

Parametric vs. Contextual Knowledge Conflict A critical failure occurs when the LLM’s parametric knowledge directly conflicts with the information found in the retrieved documents [87]. If the retrieved document states “The Austrian Chancellor is Christian Stocker,” but the LLM’s training weights strongly believe it is “Karl Nehammer,” the model may prioritize its internal, outdated weights over the retrieved information.

Citation Hallucination GR systems often aim to support verifiability by generating citations, but they frequently fall victim to citation hallucination [89]. The model may claim "The sky is blue [Doc 2]," while the retrieved

“Document 2” does not actually mention the sky’s color. Such errors erode user trust.

Evaluation & Meta-Concerns

Annotation Issues The reliability of GR evaluation depends on the quality of GT annotations in benchmark datasets. When those contain annotation errors, such as relevant passages incorrectly labeled as negative or vice versa, the measured performance metrics no longer accurately reflect a system’s true retrieval capabilities. These can lead to misleading conclusions being published that do not generalize to real-world scenarios or accurately represent comparisons.

For QA tasks, multiple passages may contain semantically equivalent answers, but only a subset are annotated as ground truth. The authors of MS MARCO, TriviaQA, and NQ each acknowledge this concern [40, 71, 25].

During our analysis, we identified such cases where the annotation can cause misleading evaluation results. Here we present three such examples: **Example 1: “which apostle had a thorn in his side” (answer: Paul).** The ground-truth passage is from the Wikipedia article “Thorn in the flesh,” which explains that Paul the Apostle used this phrase in 2 Corinthians 12:7–9. SEAL’s top-1 result was from Galatians 3, which mentions “Paul the Apostle” as the author of that epistle but contains nothing about a thorn. Even though SEAL seemed to be able to identify “Paul” as a relevant term, potentially from BART’s parametric knowledge, Galatians 3 is the wrong chapter entirely. SEAL matched on generic terms (“Paul,” “apostle”) but never generated n-grams containing “thorn” for the top-1 result.

Example 2: “how many judges currently serve on the supreme court” (answer: nine). The ground-truth passage directly states the current composition of the Court. SEAL’s top result is a historical passage stating that in 1869, “the Circuit Judges Act returned the number of justices to nine, where it has since remained.” This is borderline because the reader could infer the answer, but the passage is not specifically about the current number of judges.

Example 3: “who is the youngest judge currently sitting on the u.s. supreme court” (answer: Neil Gorsuch). The ground-truth passage discusses Gorsuch’s nomination process and him being the youngest sitting justice. SEAL’s top result directly states: “Neil Gorsuch is the youngest justice sitting, at 50 years of age.” This is a success, because SEAL found a passage that answers the question, even if not annotated.

Interpretability The lack of interpretability in generation processes presents challenges for understanding and debugging failures. Unlike sparse retrieval, where term matching is transparent [3, 4], or dense retrieval, where embedding similarity can be analyzed [8], the internal decision processes of generative retrieval are significantly more opaque [1, 90].

5 RQ2: Analysis of failures in n-gram-based GR models

In this second half of the thesis, we will conduct a high-level analysis on GR model based on n-gram [docids](#). These are a specialized group of GR models that are theoretically more robust than single docids, rely less on metadata, and provide better semantics than unstructured integers. However, these also show failure modes, which we will analyze below. To maintain the broad scope over many failure modes in this thesis, we focus on the identification of correlations to illustrate systemic trends, rather than pursuing low-level causal attribution.

For our analyses, we ran [SEAL](#) and [MINDER](#) on [NQ](#), as well as [MINDER](#) on [MSMARCO](#). Considering that MINDER uses SEAL under the hood, we expect SEAL’s results on MSMARCO to be comparative and therefore refrain from running it on MSMARCO to save space and compute.

Non-discriminative DocIDs

First and foremost, we wanted to analyze whether [MINDER](#) is relying on output volume and heuristic post-hoc intersection rather than precise docid generation. We did so by analyzing [MINDER](#) on [NQ](#) and [MSMARCO](#), alongside [SEAL](#) on [NQ](#). Unlike [NQ](#), which is based on cleanly structured Wikipedia passages with clear, discriminative metadata (titles), [MSMARCO](#) comprises web search results where metadata is famously messy, incomplete, or entirely missing, making it a useful benchmark for evaluating true retrieval robustness.

We analyze how discriminative each [docid](#) is by using the n-gram corpus frequency. This measures the “non-unique identifiers” failure mode identified in Section 4. Unlike representation failures where distinct semantic concepts are mistakenly mapped to the same [docid](#), this failure mode manifests in n-gram based [docids](#) via the model generating structurally valid n-gram identifiers that are simply too generic to uniquely isolate a single document. If the scoring function fails to penalize high-frequency terms relative to the

model’s confidence, the model might get stuck in a local optimum of generating generic words. This is an issue, as rare n-grams provide stronger evidence for specific passages, while common n-grams match thousands of topically-related ones, which dilute the score, and make the model fall back on triangulating the answer from several common n-grams instead of directly generating its identifier.

Across 6,515 queries for **NQ** and 6,980 queries for **MSMARCO**, the average corpus frequency of the top-5 scored n-grams for the top-1 retrieved passage is 28,753 for **SEAL** (NQ), 21,310 for **MINDER** (NQ), and 42,380 for **MINDER** (MSMARCO). The mean frequency across *all* generated n-grams for such passages is 382,136, 498,399, and 476,277, respectively. $k = 5$ was arbitrarily selected as a representation of the n-grams with the strongest influence on the passage’s overall score. The deciles signify the mean frequency of the top-5 n-gram for the highest-ranked retrieved queries. We chose this to ensure that the deciles capture the overall retrieval “ethos” of each query. Individual misrankings do not meaningfully affect the decile distribution.

We present the evaluation metrics in Table 2 and Table 3.

Table 2: Hits@ k success rate by top-5 n-gram frequency deciles.

Decile	SEAL (NQ)	MINDER (NQ)			MINDER (MSMARCO)		
	H@1 / H@10	H@1	H@10	H@100	H@1	H@10	H@100
D1	71.0% / 92.8%	76.5%	93.9%	99.5%	31.4%	79.1%	94.9%
D2	51.5% / 85.1%	65.9%	89.4%	98.0%	15.0%	55.8%	87.6%
D5	43.0% / 76.3%	45.8%	80.2%	95.4%	9.5%	37.6%	71.4%
D8	31.4% / 70.4%	36.7%	72.1%	90.2%	7.3%	29.4%	66.2%
D10	25.9% / 65.8%	26.8%	60.7%	82.1%	1.3%	17.6%	75.6%

Table 3: Ranking performance metrics for **MINDER** by top-5 n-gram frequency deciles.

Decile	MINDER (NQ)					MINDER (MSMARCO)				
	MRR	R@10	R@100	NDCG@10	NDCG@100	MRR	R@10	R@100	NDCG@10	NDCG@100
D1	0.828	0.597	0.831	0.626	0.687	0.469	0.785	0.949	0.536	0.573
D2	0.741	0.510	0.788	0.535	0.607	0.281	0.550	0.876	0.332	0.403
D5	0.574	0.406	0.705	0.402	0.478	0.182	0.368	0.714	0.213	0.284
D8	0.486	0.404	0.659	0.364	0.427	0.147	0.294	0.662	0.169	0.263
D10	0.382	0.351	0.607	0.299	0.366	0.079	0.176	0.756	0.083	0.207

In D1, which contains retrievals where the top-scored n-grams are highly discriminative, hit rate is excellent across the board. However, as frequency increases towards D10, which contains highly generic n-grams, ranking quality collapses. For **MINDER** on **NQ**, MRR drops from 0.828 to 0.382, and NDCG@10 plummets from 0.626 to 0.299.

The degradation is remarkably severe on **MSMARCO**. Because the model lacks clean metadata as a heuristic to anchor its search, relying on high-frequency unigrams causes Hits@1 to drop to a mere 1.3%, and MRR falls to 0.079. However, Hits@100 remains relatively high (82.1% for NQ; 75.6% for MSMARCO). This proves that the **GR** model hasn’t completely failed to match the query to the document space and the relevant passage is still retrieved in the broader candidate pool, possibly due to triangulating via the matched n-grams.

This is strongly supported by Spearman correlations: average top-5 n-gram frequency is inversely correlated with MRR ($\rho = -0.315$ for **MINDER** NQ; $\rho = -0.282$ for **MINDER** MSMARCO, $p < 0.001$) and NDCG@10 ($\rho = -0.284$ and $\rho = -0.318$, respectively). Notably, using average frequency across *all* n-grams shows no meaningful correlation with performance, suggesting that the presence of highly-ranked generic terms, rather than overall volume of “genericity”, drives the ranking collapse and that the model identifies the ground truth passages relatively successfully, even if it cannot generate highly specific, discriminative n-grams. **MINDER** does not inherently penalize high-frequency n-grams, instead relying on its artificially boosted pseudoqueries to push documents to the top [24]. When those fail to provide specificity, the model’s ranking ability fails with it.

Length Bias Given that shorter n-grams naturally tend to be much more frequent than longer ones, this failure mode is closely tied to the issue of non-discriminative **docids**. That said, we examine whether the analyzed systems disproportionately generate short n-grams, such as unigrams and bigrams, rather than longer, more distinctive multi-word sequences. Longer n-grams generally provide stronger discriminative power, as multi-token strings are far less likely to appear across multiple irrelevant passages.

We quantify this scoring bias by measuring the fraction of generated n-grams that consist of a single token for the top-retrieved passages. Across all queries, the mean unigram proportion is 0.869 ± 0.066 for **SEAL** (NQ), 0.853 ± 0.071 for **MINDER** (NQ), and 0.855 ± 0.106 for **MINDER** (MSMARCO). The average absolute n-gram length is 1.28 words for **SEAL**, 1.48 for **MINDER** NQ, and 1.45 for **MINDER** MSMARCO. This indicates that more than four-fifths of generated n-grams are unigrams.

We present our results in Table 4 and Table 5.

Table 4: Hits@ k success rate by unigram proportion deciles.

Decile	SEAL (NQ)	MINDER (NQ)			MINDER (MSMARCO)		
	H@1 / H@10	H@1	H@10	H@100	H@1	H@10	H@100
D1	67.7% / 91.2%	73.6%	92.8%	98.5%	27.0%	70.8%	91.3%
D2	61.6% / 87.1%	68.2%	92.2%	98.2%	18.4%	51.6%	82.7%
D3	48.8% / 82.5%	57.8%	85.6%	96.7%	14.5%	47.0%	78.3%
D5	44.2% / 81.0%	48.0%	81.9%	95.5%	11.2%	41.4%	71.9%
D8	28.4% / 69.9%	31.6%	73.7%	92.7%	6.9%	29.8%	63.4%
D10	22.6% / 60.5%	20.7%	55.9%	79.9%	0.3%	15.6%	84.4%

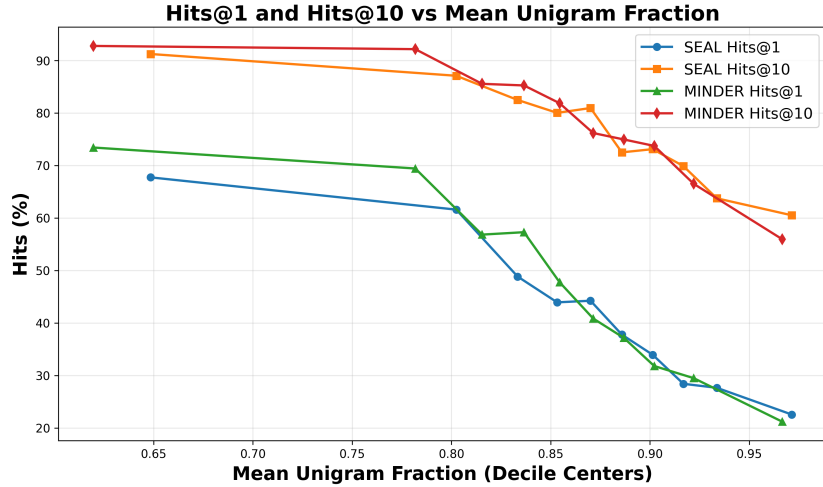
Table 5: Ranking performance metrics for MINDER by unigram proportion deciles.

Decile	MINDER (NQ)					MINDER (MSMARCO)				
	MRR	R@10	R@100	NDCG@10	NDCG@100	MRR	R@10	R@100	NDCG@10	NDCG@100
D1	0.806	0.539	0.985	0.582	0.655	0.417	0.701	0.913	0.475	0.523
D2	0.765	0.522	0.982	0.554	0.623	0.297	0.508	0.827	0.336	0.403
D3	0.667	0.465	0.967	0.479	0.551	0.246	0.463	0.783	0.285	0.353
D5	0.594	0.444	0.955	0.430	0.503	0.206	0.403	0.719	0.242	0.308
D8	0.459	0.400	0.927	0.350	0.431	0.142	0.296	0.634	0.168	0.247
D10	0.327	0.305	0.799	0.255	0.328	0.074	0.155	0.844	0.070	0.200

For MINDER (NQ), queries with the highest proportion of unigrams (D10) suffer an MRR drop of 0.479 compared to D1. On MSMARCO, the penalty is incredibly severe, queries over-reliant on unigrams achieve a near-total failure rate in top positioning (0.3% Hits@1, 0.070 NDCG@10). However, once again, Hits@100 remains at 84.4% for MSMARCO in D10, showing again that the generative model knows the semantic neighborhood of the answer, but evaluating documents via nondistinct, single tokens means the model cannot use the context of the passage required to rank the correct passage at rank-1.

The Spearman ρ confirms that this negatively impacts ranking quality: for MINDER (NQ), unigram proportion vs. MRR is $\rho = -0.351$, and vs. NDCG@10 is $\rho = -0.304$. For MINDER (MSMARCO), it is $\rho = -0.267$ and $\rho = -0.323$ ($p < 0.001$).

To understand the results more illustratively, we summarized the trend in Figure 5, showing our results for SEAL and MINDER on the NQ dataset. We can see a convergence in hit rates as unigram proportions increase. Without multi-token strings to enforce specific intersections, generative systems struggle to confidently disambiguate documents.



Metadata Dependency

Both [SEAL](#) and [MINDER](#) rely on passage metadata heavily to retrieve the correct passages, which can be considered a form of representation learning failure. We analyzed this for SEAL and MINDER on [NQ](#) only, as this dataset enables the models to rely on metadata. Comparable with the previous section, we can expect that the model qualities drop on MSMARCO. For [SEAL](#), in the top-100 retrieved passages among the 6515 queries analyzed, titles account for 39.85% of the total score. For [MINDER](#), titles make up 29.6% and pseudoqueries 19.67% of the total score. Interestingly, this overreliance drops when we only account for successfully retrieved passages, with titles accounting for 29.1% for [SEAL](#) and 20.6% for [MINDER](#) (11.27% for pseudoqueries). In cases where the answer string is found exactly both as a “title n-gram” and “nontitle n-gram”, the title n-gram receives on average 2866% more score in [SEAL](#) and 2499% in [MINDER](#). The scoring differential between the answer string being a “title n-gram” and “nontitle n-gram” also drops to 1876% for [SEAL](#) and 1439% for [MINDER](#) when only accounting for successfully retrieved passages. The scoring differential between title n-grams and all other n-grams generated is 521% for [SEAL](#), and 566% for [MINDER](#) (for pseudoqueries 197%). This can indicate that when the model is “correct”, it is using more of the body-text, but also that both models are overreliant on metadata. This can be considered a “feature” on cases where there is clear metadata in the dataset (e.g. [NQ](#)) and on cases where retrieving the correct document is more important than retrieving the current passage. However, if our aim is to create a generalized GR model, this should be considered a “failure”.

Both Bevilacqua et al. [18] and Li et al. [24, 35, 36] note that performance

drops when titles are unavailable [18, 24, 35, 36]. Bevilacqua et al. [18], however, does not report **SEAL** ablation results without titles as identifiers. **MINDER** performs 10.3% worse recall@5 and 7.4% worse recall@20 on NQ without titles and pseudoqueries. On **MSMARCO** [40], **SEAL** had a 30.7% worse Recall@5 than BM25 on **MSMARCO**, with **MINDER** only marginally (3.1%) better than BM25, likely due to pseudoqueries. Neither **SEAL** nor **MINDER** achieved a Recall@5 larger than 30% on **MSMARCO**. LTRGR and DGR achieved 40.2% and 42.9%, respectively. It is also worth noting that NQ is based on highly structured Wikipedia passages, while MSMarco is less cleanly structured with messier or lacking metadata.

Our analysis shows that for **SEAL**, the title score is higher by around 280% on average for positive “ground truth” retrieved passages than negative ones. However, the median is only around 41% and the σ around 1113%, indicating that there is a subset of queries where the title is massively scored, indicating that the model uses it as quasi the only identifier, while for others the title is much less helpful. For **MINDER**, accordingly, the values are 325%, 57.5%, 939%. An extreme example was found in **SEAL**, where for the query “who sang this is how we do it”, correct passages have 545x more title score than incorrect passages (in passages where a title-as-ngram is present). In these cases, title matching is extremely discriminative, which makes it well suited for datasets with clear metadata, but potentially bad for ones without.

To strengthen this conclusion, we analyze whether retrieval success correlates with the model’s ability to generate the specific answer string as an identifier or whether it relies on the surrounding context. While generating the answer string provides a valuable anchor for a passage, the primary goal of **GR** is document identification, not direct question answering. This analysis is presented for NQ only, as MSMARCO’s incomplete metadata makes answer string matching unreliable and complex.

Table 6 illustrates the results of our analysis. We looked at whether among the generated n-grams for the top-1 and top-10 scored retrieved passages at least one of them contained the answer string and what the Hits@1 value is for these cases. The results are mostly similar between **SEAL** and **MINDER**, with the generated n-grams of most top-1 ranked retrieved passages not containing the answer string, yet when they do, the respective query’s Hits@1 rate doubles and Hits@10 increases significantly.

We can see that in cases where the answer string is identified among the n-grams, the success rate jumps significantly for both systems. This indicates that direct identification is a strong predictor of retrieval success. Both **SEAL** and **MINDER** are able to retrieve (at least) one correct passage in around 35% of cases in the first place and 73% of cases among the top-10 retrieved, suggesting that the model is able to identify the semantic environment in

Table 6: Hitrate conditioned on answer string presence on NQ.

Metric	SEAL		MINDER	
	Pres.	Abs.	Pres.	Abs.
Queries (N)	1,096	5,419	1,457	5,058
Hits@1	75.0%	34.9%	75.4%	38.2%
Hits@10	93.5%	72.6%	93.0%	74.3%

which an answer is likely to be in relatively convincingly. That said, the performance gap between the two cases indicates that when the model fails to generate the specific answer as an identifier, the remaining contextual cues are oftentimes too generic to distinguish the correct passage from topically similar alternatives.

Similarly, we analyze the contribution of synthetic identifiers in the **MINDER** framework. As detailed in Section 2.2, **MINDER** incorporates pseudo-queries alongside titles and substrings to capture high-level semantic information. Our analysis of 6,515 queries reveals an interesting divergence between the quantity of generated pseudo-queries and their influence on the final ranking.

Numerically, pseudo-queries on **NQ** constitute a negligible fraction of the generated identifiers, accounting for only 0.32% of the total n-grams (87714 out of 26.9 million). On **MSMARCO**, they constitute 0.37%. However, their contribution to the total accumulated score is disproportionately high, representing 4.52% of the total score mass (14.8 million out of 328.5 million). On **MSMARCO**, this percentage is 7.30%. This confirms that **MINDER**’s scoring mechanism successfully uses sparse but highly discriminative synthetic identifiers.

Interestingly, titles still dominate the picture the scoring disproportionately, as only 1.08% of ngrams are titles yet contribute 23.2% of the total score. For **MSMARCO**, titles constitute 0.19% of ngrams and are worth 4.22% of the total score on **MINDER**.

Cultural Bias

For **SEAL**, this is best illustrated by the query: “where did the southern song have their capital” from the NQ dataset.

Although the correct answer is Lin’an (the query referring to the Southern Song Dynasty), **SEAL** focused entirely on passages about the southern United States, especially about Tennessee and Virginia. The model likely found a quasi-perfect “query keyword match” for such passages and connected

“southern” with the southern US, “song” with country music, the ‘Dixie’ song and the ‘Music City’ sobriquet of Nashville, Tennessee, and, building onto these, “capital” with Nashville being the capital of Tennessee and Richmond being the capital of Virginia and the former capital of the confederate South. These substrings, while common, were likely propped up by their implicit connection to the US in the dataset and BART, even though they possess different meanings in a US geography context versus a 12th-century Chinese history context.

There are no ground truth passages retrieved in the *top-100* retrieved passages, which consist entirely of passages about the southern United States. This is also an example of *false pruning*, as SEAL seemingly discarded the tokens related to the Song Dynasty early on. This example can also be considered a failure of term disambiguation.

An important observation is that MINDER successfully retrieved the relevant passages. All queries in the *top-100* retrieved passages were in some way relevant to Ancient China, with the first ground-truth passage appearing at rank-4. Both systems use BART-large as their autoregressive language model and the same dataset to build their FM-index. The observed success is therefore likely due to the use of pseudoqueries. Since BART is capable of generating the correct terms, failure in SEAL likely happened due to failing to disambiguate the context, in which pseudoqueries assisted. For the highest-ranked retrieved GT passage, MINDER generated pseudoqueries such as *what was the capital of the song dynasty in north china* and *who took over the capital of the song dynasty*.

Similar issues can be identified for the following queries: “who was the last king of scotland based on” The correct answer is Idi Amin, a former Ugandan President. SEAL retrieved passages entirely about Scottish heads of state. For the query “what country on chinas border is mostly desert”, SEAL’s top results include Pakistan, Russia, and North Korea, which are not technically correct. These countries have a disproportionate amount of coverage from a Eurocentric point-of-view compared to the obvious answer, Mongolia.

Similar questions can be raised for discriminative language, stereotyping, toxicity, and other forms of culture-related biases [73].

Query-to-DocID Overlap As explained previously, disambiguating the query remains a challenging task. Additionally, making sure the model “understands” the query instead of just heuristically generating the tokens found in the query is a challenge that would improve GR’s robustness to queries with synonymous or near-match terms. We measure the lexical align-

ment between query terms and the n-grams. High overlap indicates that the model successfully generated n-grams containing query-relevant terms, while low overlap suggests a mismatch where generated n-grams, though potentially topically related, do not directly address the query’s specific terms. We quantify this using the fraction of query tokens that appear in any generated n-gram in the highest-scored retrieved passage, denoted by $C(Q, G_n)$, where Q denotes the query tokens and G_n denotes the tokens of the generated n-grams.

Our analysis across 6,515 queries shows that the token-level lexical overlap varies substantially across the top-1 retrieved passages. As visible in Table 7, there is a positive correlation between lexical overlap and retrieval success. For **SEAL**, $\rho = 0.323$ for $k = 1$ and $\rho = 0.190$ for $k = 10$. For **MINDER**, the Spearman correlations are $\rho = 0.380$ and $\rho = 0.202$, respectively for NQ and $\rho = 0.223$ and $\rho = 0.231$ for MSMARCO. All values are statistically significant at $p < 0.001$.

Table 7: Hits@ k by $C(Q, G_n)$ buckets.

Bucket	Range $C(Q, G_n)$	SEAL	MINDER (NQ)	MINDER (MSMARCO)
		H@1 / H@10	H@1 / H@10	H@1 / H@10
B1	0.00–0.20	9.9% / 44.7%	6.7% / 31.7%	0.4% / 19.1%
B2	0.20–0.40	23.9% / 62.4%	18.3% / 60.3%	2.9% / 23.4%
B3	0.40–0.60	43.6% / 80.1%	36.5% / 74.4%	5.3% / 30.3%
B4	0.60–0.80	58.3% / 87.3%	53.2% / 84.1%	12.5% / 42.2%
B5	0.80–1.00	76.8% / 94.2%	74.7% / 92.5%	21.1% / 57.7%

As visible from Table 7, the hit rate of **SEAL** and **MINDER** are relatively similar, and are positively correlated with $C(Q, G_n)$. Similar to other failure modes, low query coverage typically arises when the model generates semantically related but lexically distinct n-grams (e.g., for a query containing “automobile,” the model could generate n-grams with “car”) or when the model generates generic topical n-grams that lack the specific query terms needed to isolate the answer (e.g., a query about “Victorian architecture in London” generates only “architecture” and “London”).

Overreliance on a Single Identifier Here we measure whether badly scored retrievals rely on one big “confidently incorrect” score, as both **SEAL** and **MINDER** are designed to work by aggregating signals across multiple n-grams. We define this as the ratio of the highest scored n-gram score and the sum of all matched n-gram scores for the passage. Results and analysis for MSMARCO are omitted to keep the analysis in scope. The patterns follow

those observed in the non-discriminative DocID and length bias analyses above.

Our analysis across 6,515 queries reveals that a lower such ratio correlates with higher retrieval success, as shown in Table 8. For **SEAL**, we observe a Spearman correlation of $\rho = -0.115$ ($p < 0.001$), with success rates declining from 48.6% in the lowest decile (D1) to 29.6% in the highest decile (D10).

Table 8: Hits@1 on NQ by top n-gram ratio deciles (max score / sum scores).

Decile	SEAL (Mean: 0.44)		MINDER (Mean: 0.36)	
	Range	Hits@1	Range	Hits@1
D1	0.12–0.30	48.6%	0.12–0.25	58.0%
D2	0.30–0.35	45.8%	0.25–0.28	59.3%
D3	0.35–0.38	45.4%	0.28–0.31	55.8%
D5	0.41–0.44	44.3%	0.33–0.35	45.1%
D8	0.50–0.53	35.4%	0.40–0.43	40.2%
D10	0.57–0.85	29.6%	0.48–0.94	32.5%

This trend is also noticeable in **MINDER** ($\rho = -0.169$, $p < 0.001$), which also exhibits a lower mean ratio of 0.36 compared to **SEAL**’s 0.44 and a higher hit rate in all deciles. This shift toward more distributed evidence is assumed to be a consequence of **MINDER**’s multiview identifiers, since by aggregating scores from titles, pseudo-queries, and substrings, **MINDER** theoretically reduces reliance on any single identifier.

We also found that failures are slightly more heterogeneous in terms of their distribution among “top-ngram score ratio” values. However, this difference is small, as the standard deviation of this ratio among failure cases is roughly 5-8% larger than that among the success cases. Success was measured as Precision@1.

This finding aligns with the additive scoring design, which benefits from distributed evidence by providing redundant confirmation across multiple n-grams. Conversely, if the top-scoring n-gram matches many passages or identifies the wrong one, the remaining low-scoring n-grams may provide insufficient corrective evidence to ensure correct ranking.

Overlapping N-gram Tokens We initially hypothesized that cases where the model produces semantically redundant n-grams with high token overlap would constitute a failure mode by inflating scores without providing new evidence. To test this, we measure a ratio defined as the number of unique tokens divided by the total number of tokens across all generated

n-grams for the top-1 retrieved query, which we will refer to as “diversity”. This analysis is presented for NQ only. Comparable trends are observed on MSMARCO.

Table 9 illustrates the results on the NQ dataset. Contrary to our initial hypothesis, the results reveal a negative correlation between diversity and hits@k. For **SEAL**, $\rho = -0.111$ for $k = 1$ and $\rho = -0.101$ for $k = 10$. For **MINDER**, the Spearman correlations are $\rho = -0.289$ and $\rho = -0.242$, respectively. All values are statistically significant at $p < 0.001$.

Table 9: Hits@k by token-based diversity ratio deciles (unique / total tokens).

Decile	Div.	SEAL (NQ)			Div.	MINDER (NQ)		
		H@1	H@10	H@100		H@1	H@10	H@100
D1	0.36–0.48	52.6%	84.4%	96.0%	0.28–0.56	70.9%	91.4%	98.8%
D2	0.48–0.49	46.3%	81.4%	95.4%	0.56–0.64	64.3%	88.6%	97.7%
D3	0.49–0.50	43.1%	78.2%	94.4%	0.64–0.71	53.2%	85.6%	96.3%
D4	0.50–0.51	49.5%	82.2%	94.9%	0.71–0.76	51.8%	83.7%	95.4%
D5	0.51–0.52	40.4%	73.9%	93.5%	0.76–0.79	53.7%	84.1%	96.8%
D6	0.52–0.53	41.3%	76.4%	94.1%	0.79–0.82	45.0%	78.9%	95.1%
D7	0.53–0.53	39.2%	73.1%	90.0%	0.82–0.85	41.9%	78.1%	96.3%
D8	0.53–0.54	34.9%	68.1%	89.3%	0.86–0.88	32.2%	73.2%	91.9%
D9	0.54–0.56	37.9%	74.5%	89.1%	0.89–0.92	30.0%	65.4%	87.2%
D10	0.56–0.63	32.4%	69.8%	88.2%	0.92–1.00	22.0%	55.6%	78.8%

Decile	MINDER (MSMARCO)				
	Div.	H@1	H@10	H@100	
D1	0.14–0.33	27.1%	74.9%	92.1%	<i>Note: Bins are constructed as equal-count deciles. Each bin contains $N \approx 650$ queries for NQ and $N \approx 698$ for MSMARCO.</i>
D2	0.33–0.37	17.8%	55.6%	86.4%	
D3	0.37–0.41	14.2%	45.9%	79.1%	
D4	0.41–0.43	11.1%	42.1%	74.9%	
D5	0.43–0.45	10.9%	37.2%	71.0%	
D6	0.45–0.47	9.0%	34.2%	68.1%	
D7	0.47–0.49	8.3%	31.7%	64.5%	
D8	0.49–0.50	4.2%	20.7%	67.7%	
D9	0.50–0.50	0.0%	15.9%	80.5%	
D10	0.50–0.64	5.0%	33.0%	69.2%	

It is interesting to note that **MINDER** has a considerably higher success rate at low diversity scenarios. This is expected, as **MINDER** generates titles, pseudo-queries, and substrings simultaneously, meaning that the model frequently repeats key tokens across these different identifier views. The highest success rate is found in the lowest diversity decile, which suggests that the model is able to reinforce a specific semantic cluster through multiview identifiers.

These findings suggest that token repetition is not a redundancy failure, but rather a signal of model certainty. When **SEAL** and **MINDER** are highly confident in a target document, it generates multiple overlapping n-grams that concentrate score on a specific semantic cluster. In contrast, high identifier diversity indicates a lack of model confidence, where the model produces a wide variety of topically related but lexically distinct tokens that fail to converge.

However, it is worth noting that the results for **MINDER** on MSMARCO are notably lower than on NQ for Hit@1 and Hits@10, indicating that pinpointing the specific answer is challenging when the dataset is messier. Similarly, as expected, diversity ratio ranges are also lower on MSMarco, indicating that the dataset is less standardized. Interestingly, the Hits@100 rate on MSMarco is comparable to that on NQ, suggesting that even if the model struggles to retrieve the exact answers on the top, the model can still triangulate at least one correct passage in the top-100 retrieved results in most cases.

Unique Articles among Passages We analyzed whether there is a correlation between retrieval failure and the number of unique titles retrieved in the top-k passages. For this, according to the other analyses, we use $k = 10$. This analysis is possible as Wikipedia articles have unique titles. We present our results for the NQ dataset. The results on the MSMarco dataset for **MINDER** are omitted from this thesis to keep it in scope but can be computed using the scripts available under the [GitHub repository](#)² of this thesis.

The results are illustrated in Table 10 and Figure 2.

Table 10: Hits@k by title repetition count on NQ.

Max Repetition *	SEAL		MINDER	
	<i>N</i>	H@1 / H@10	<i>N</i>	H@1 / H@10
1	23	30.4% / 60.9%	38	28.9% / 60.5%
2	556	45.3% / 70.1%	665	46.9% / 72.9%
4	653	42.1% / 77.3%	819	46.8% / 77.7%
6	573	41.5% / 76.1%	650	45.1% / 79.5%
8	567	44.3% / 78.0%	609	49.8% / 82.8%
10	1,606	38.0% / 76.7%	990	42.1% / 79.1%

*Maximum number of passages sharing the same title in the top-10 results.

²<https://github.com/richietk/thesis/>

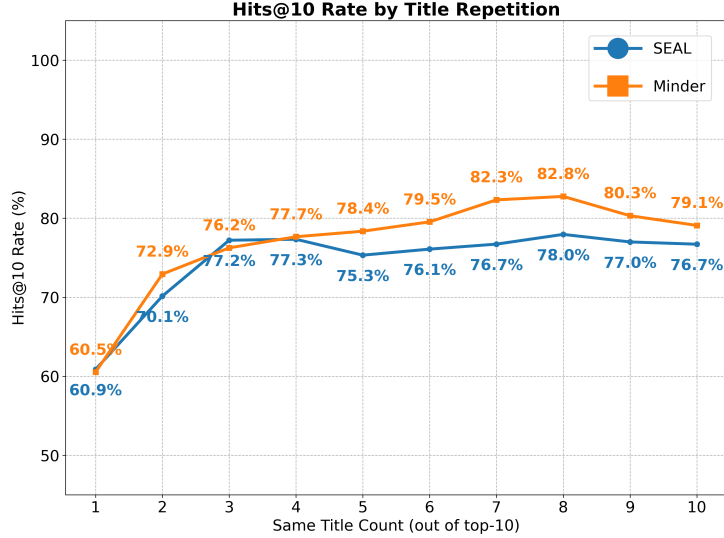


Figure 2: Hits@k by maximum title repetition count in the top-10 results on NQ for SEAL and MINDER.

We can see that when stratified by maximum title repetition, i.e. the maximum number of passages from the same Wikipedia article per query, **MINDER** marginally outperforms **SEAL**, especially on queries where the system is relatively sure of the answer, as evidenced by a specific title appearing in several of the top-10 results. We can see that in cases where the max repetition is very low, i.e. the model couldn’t confidently pinpoint a single document and is likely guessing, retrieval success suffers. Based on this analysis, there are significantly more cases with high title repetition than low title repetition, indicating that the models are fairly certain about the correct document (or, alternatively, are confidently incorrect about one).

6 Discussion

Tracing Artifacts to Root Mechanisms

The retrieval “symptoms” observed in the scoring and ranking behavior of SEAL and MINDER can be traced back to several upstream failure modes.

The significant drop in Hits@1 when identifiers lack lexical specificity can be traced back to the failure mode when the corpus size exceeds the transformer’s parametric memory, forcing the model to default to high-frequency linguistic priors rather than discriminative document mappings. As the corpus scales, the model loses the fine-grained resolution required to distinguish

unique identifiers, leading to an overreliance on generic terms that match many topically-related passages.

The disproportionate impact of answer-string presence on retrieval success reflects a reliance on rote memorization and a dependence on metadata, as GR models often overfit to their training data [23], dropping performance when queries are not phrased the way the model saw queries during training. When that anchor is absent, the model cannot generalize enough to discriminate the context. This failure also suggests that a passage’s “semantic environment” is too generic to discriminate each passage individually.

Failure Mode Categorization by Downstream Task

As explained by Li et al. [16], GR has mostly been applied and tested KILT [91] tasks, such as fact verification, open-domain QA, conversational QA, entity linking, multi-hop retrieval, recommender systems, and code retrieval.

While our taxonomy identifies failures common to the GR paradigm, aiming to explain them in a task-agnostic way, certain failure modes may be more or less severe depending on the downstream task:

- Indistinct identifiers serve as a critical failure mode in entity linking when terms are ambiguous. As an example, IVF could mean both *Inverted File Index* and *In vitro fertilization*.
- Vocabulary/tokenization issues are exacerbated in code retrieval scenarios, as splitting tokens may break semantic units in programming language syntax

Empirical validation of these task-specific effects requires dedicated experiments beyond the scope of this work.

Limitations

The empirical analysis is limited to open-domain QA of SEAL and MINDER. In this, two specific limitations must be mentioned. Firstly, we have not tested the aforementioned models on other KILT tasks, such as dialogue generation, fact checking, or slot filling. Secondly, several other published GR models have been only analyzed qualitatively via the Literature Review section of our work. While we believe the identified failure modes generalize to other GR frameworks and use-case applications, their task- and model-specific manifestations require dedicated studies and are a potential future research direction.

Additionally, it is worth noting that several of the analysis results were presented only for the NQ dataset where the results for MSMarco were comparable. This is deliberately done to keep the thesis in scope and comply with the length limits of a Bachelor’s thesis.

Future Work

Future work could be conducted on several areas of research that have been out-of-focus for this thesis. First and foremost, as this thesis focused on the “retrieval” part of GR, a systematic classification of the failure modes in response generation is warranted. Nonetheless, as that area of GR heavily overlaps with RAG, such analyses have already been conducted to some extent [87, 88]. Secondly, a more detailed analysis of failure modes in multimodal GR models, such as those capable of processing and returning audiovisual input, would be a useful direction, as these models exhibit unique failure modes not present in text-based information retrieval. Similarly, experiments regarding downstream task-specific effects, such as failure modes in GR for code retrieval or entity linking, are also beyond scope, as this thesis aimed to give a general summary of failure modes applicable to most GR architectures. Failure modes in multilingual GR systems [92] could also be explored, as there has been little published work on GR performance on non-English corpora. Future work could also focus on implementing recent advances in neural network architectures, such as RetNet [79] and Mamba [80], as a move away from Transformer-based architectures to improve the versatility and efficiency of future GR systems. Skip-gram based docids could also be explored, though they come with their own set of challenges, which are also out-of-scope for this thesis.

Additionally, of the failure modes addressed in this thesis, the challenges of continual learning, permutation-invariant decoding, and robustness to zero-shot retrieval are worth highlighting, as these areas have massive potential upside if fully solved, yet continue to pose unique challenges, such as the questions highlighted in this thesis for dynamic corpora.

7 Conclusion

This thesis has investigated the failure modes of GR (Generative Retrieval), which moves beyond the traditional “index-retrieve-then-rank” paradigm, to evaluate the robustness of such systems. Via combining an extensive literature review and an empirical analysis of the SEAL and MINDER architectures on the NQ dataset, we have addressed the most important limitations

of the [GR](#) framework and developed a taxonomy.

In response to **RQ1**, we developed a failure taxonomy that categorizes [GR](#) limitations across several distinct stages: DocID design, training, decoding, scoring, evaluation, and response generation, as well as “meta” concerns. This classification showed that [GR](#) failures are much more complex than they seem and often stem from structural limitations and design choices.

Our quantitative investigation into **RQ2** aimed to quantify the extent of several failure modes by directly analyzing the outputs of two foundational [GR](#) models, [SEAL](#) and [MINDER](#). Accordingly, we analyzed over 6500 queries, totaling more than half a million retrieved passages analyzed. We identified several “symptoms” of retrieval failures, such as the over-reliance on metadata like titles, and the problem of autoregression that longer [docids](#) get lower scores.

While [GR](#) presents a promising “all-in-one” architecture that reduces the need for external vector indexes, memory, and theoretically improves semantic awareness in retrieval, it remains “brittle” in its current state, with solutions to many of its failure modes not being fully achieved to this day.

References

- [1] Yubao Tang, Ruqing Zhang, Weiwei Sun, Jiafeng Guo, and Maarten De Rijke. Recent advances in generative information retrieval. In *Companion Proceedings of the ACM Web Conference 2024*, WWW '24, page 1238–1241, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400701726. doi: 10.1145/3589335.3641239. URL <https://doi.org/10.1145/3589335.3641239>.
- [2] Donald Metzler, Yi Tay, Dara Bahri, and Marc Najork. Rethinking search: making domain experts out of dilettantes. *ACM SIGIR Forum*, 55(1):1–27, June 2021. ISSN 0163-5840. doi: 10.1145/3476415.3476428. URL <http://dx.doi.org/10.1145/3476415.3476428>.
- [3] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3:333–389, 01 2009. doi: 10.1561/15000000019.
- [4] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. Splade: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '21, page 2288–2292, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450380379. doi: 10.1145/3404835.3463098. URL <https://doi.org/10.1145/3404835.3463098>.
- [5] Eunseong Choi, Sunkyung Lee, Minjin Choi, Hyeseon Ko, Young-In Song, and Jongwuk Lee. Spade: Improving sparse representations using a dual document encoder for first-stage retrieval. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, CIKM '22, page 272–282. ACM, October 2022. doi: 10.1145/3511808.3557456. URL <http://dx.doi.org/10.1145/3511808.3557456>.
- [6] Biplob Biswas and Rajiv Ramnath. Efficient and interpretable information retrieval for product question answering with heterogeneous data, 2024. URL <https://arxiv.org/abs/2405.13173>.
- [7] Luyu Gao, Zhuyun Dai, and Jamie Callan. Coil: Revisit exact lexical match in information retrieval with contextualized inverted list, 2021. URL <https://arxiv.org/abs/2104.07186>.
- [8] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. Dense passage

- retrieval for open-domain question answering, 2020. URL <https://arxiv.org/abs/2004.04906>.
- [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423/>.
 - [10] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval, 2020. URL <https://arxiv.org/abs/2007.00808>.
 - [11] Haitao Li, Qingyao Ai, Jingtao Zhan, Jiaxin Mao, Yiqun Liu, Zheng Liu, and Zhao Cao. Constructing tree-based index for efficient and effective dense retrieval, 2023. URL <https://arxiv.org/abs/2304.11943>.
 - [12] Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernández Ábrego, Ji Ma, Vincent Y. Zhao, Yi Luan, Keith B. Hall, Ming-Wei Chang, and Yinfei Yang. Large dual encoders are generalizable retrievers, 2021. URL <https://arxiv.org/abs/2112.07899>.
 - [13] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.703. URL <https://aclanthology.org/2020.acl-main.703/>.
 - [14] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL <http://jmlr.org/papers/v21/20-074.html>.

- [15] Peitian Zhang, Zheng Liu, Yujia Zhou, Zhicheng Dou, Fangchao Liu, and Zhao Cao. Generative retrieval via term set generation, 2024. URL <https://arxiv.org/abs/2305.13859>.
- [16] Xiaoxi Li, Jiajie Jin, Yujia Zhou, Yuyao Zhang, Peitian Zhang, Yutao Zhu, and Zhicheng Dou. From matching to generation: A survey on generative information retrieval, 2025. URL <https://arxiv.org/abs/2404.14851>.
- [17] Tzu-Lin Kuo, Tzu-Wei Chiu, Tzung-Sheng Lin, Sheng-Yang Wu, Chao-Wei Huang, and Yun-Nung Chen. A survey of generative information retrieval, 2024. URL <https://arxiv.org/abs/2406.01197>.
- [18] Michele Bevilacqua, Giuseppe Ottaviano, Patrick Lewis, Scott Yih, Sebastian Riedel, and Fabio Petroni. Autoregressive search engines: Generating substrings as document identifiers. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=Z4kZxAjg8Y>.
- [19] Hansi Zeng, Chen Luo, and Hamed Zamani. Planning ahead in generative retrieval: Guiding autoregressive generation through simultaneous decoding, 2024. URL <https://arxiv.org/abs/2404.14600>.
- [20] Weiwei Sun, Lingyong Yan, Zheng Chen, Shuaiqiang Wang, Haichao Zhu, Pengjie Ren, Zhumin Chen, Dawei Yin, Maarten de Rijke, and Zhaochun Ren. Learning to tokenize for generative retrieval, 2023. URL <https://arxiv.org/abs/2304.04171>.
- [21] Yubao Tang, Ruqing Zhang, Jiafeng Guo, Maarten de Rijke, Wei Chen, and Xueqi Cheng. Listwise generative retrieval models via a sequential learning process, 2024. URL <https://arxiv.org/abs/2403.12499>.
- [22] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, March 2023. ISSN 1557-7341. doi: 10.1145/3571730. URL <http://dx.doi.org/10.1145/3571730>.
- [23] Ronak Pradeep, Kai Hui, Jai Gupta, Adam D. Lelkes, Honglei Zhuang, Jimmy Lin, Donald Metzler, and Vinh Q. Tran. How does generative retrieval scale to millions of passages?, 2023. URL <https://arxiv.org/abs/2305.11841>.

- [24] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. Multiview identifiers enhanced generative retrieval, 2023. URL <https://arxiv.org/abs/2305.16675>.
- [25] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Jakob Dai, Andrew M. and Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL <https://aclanthology.org/Q19-1026/>.
- [26] Alec Radford and Karthik Narasimhan. Improving language understanding by generative pre-training. 2018. URL <https://api.semanticscholar.org/CorpusID:49313245>.
- [27] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, et al. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- [28] Yi Tay, Vinh Q. Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, Tal Schuster, William W. Cohen, and Donald Metzler. Transformer memory as a differentiable search index, 2022. URL <https://arxiv.org/abs/2202.06991>.
- [29] Yongjian Chen, Tao Guan, and Cheng Wang. Approximate nearest neighbor search by residual vector quantization. *Sensors (Basel)*, 10 (12):11259–11273, December 2010.
- [30] Hansi Zeng, Chen Luo, Bowen Jin, Sheikh Muhammad Sarwar, Tianxin Wei, and Hamed Zamani. Scalable and effective generative information retrieval, 2023. URL <https://arxiv.org/abs/2311.09134>.
- [31] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. Autoregressive entity retrieval, 2021. URL <https://arxiv.org/abs/2010.00904>.

- [32] Jiangui Chen, Ruqing Zhang, Jiafeng Guo, Yiqun Liu, Yixing Fan, and Xueqi Cheng. Corpusbrain: Pre-train a generative retrieval model for knowledge-intensive language tasks. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, CIKM '22, page 191–200. ACM, October 2022. doi: 10.1145/3511808.3557271. URL <http://dx.doi.org/10.1145/3511808.3557271>.
- [33] Jiangui Chen, Ruqing Zhang, Jiafeng Guo, Yixing Fan, and Xueqi Cheng. Gere: Generative evidence retrieval for fact verification. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '22, page 2184–2189. ACM, July 2022. doi: 10.1145/3477495.3531827. URL <http://dx.doi.org/10.1145/3477495.3531827>.
- [34] Yujia Zhou, Jing Yao, Zhicheng Dou, Ledell Wu, Peitian Zhang, and Ji-Rong Wen. Ultron: An ultimate retriever on corpus with a model-based indexer, 2022. URL <https://arxiv.org/abs/2208.09257>.
- [35] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. Learning to rank in generative retrieval, 2023. URL <https://arxiv.org/abs/2306.15222>.
- [36] Yongqi Li, Zhen Zhang, Wenjie Wang, Liqiang Nie, Wenjie Li, and Tat-Seng Chua. Distillation enhanced generative retrieval. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 11119–11129, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.662. URL <https://aclanthology.org/2024.findings-acl.662/>.
- [37] Noah Ziems, Wenhao Yu, Zhihan Zhang, and Meng Jiang. Large language models are built-in autoregressive search engines. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 2666–2678, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.167. URL <https://aclanthology.org/2023.findings-acl.167/>.
- [38] Yujing Wang, Yingyan Hou, Haonan Wang, Ziming Miao, Shibin Wu, Hao Sun, Qi Chen, Yuqing Xia, Chengmin Chi, Guoshuai Zhao, Zheng Liu, Xing Xie, Hao Allen Sun, Weiwei Deng, Qi Zhang, and Mao Yang. A neural corpus indexer for document retrieval, 2023. URL <https://arxiv.org/abs/2206.02743>.

- [39] Shengyao Zhuang, Houxing Ren, Linjun Shou, Jian Pei, Ming Gong, Guido Zuccon, and Daxin Jiang. Bridging the gap between indexing and retrieval for differentiable search index with query generation, 2023. URL <https://arxiv.org/abs/2206.10128>.
- [40] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. Ms marco: A human generated machine reading comprehension dataset, 2018. URL <https://arxiv.org/abs/1611.09268>.
- [41] Jiangui Chen, Ruqing Zhang, Jiafeng Guo, Maarten de Rijke, Wei Chen, Yixing Fan, and Xueqi Cheng. Continual learning for generative retrieval over dynamic corpora. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, CIKM '23*, page 306–315, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701245. doi: 10.1145/3583780.3614821. URL <https://doi.org/10.1145/3583780.3614821>.
- [42] Sunkyung Lee, Minjin Choi, and Jongwuk Lee. Glen: Generative retrieval via lexical index learning, 2023. URL <https://arxiv.org/abs/2311.03057>.
- [43] Thong Nguyen and Andrew Yates. Generative retrieval as dense retrieval, 2023. URL <https://arxiv.org/abs/2306.11397>.
- [44] Ravisri Valluri, Akash Kumar Mohankumar, Kushal Dave, Amit Singh, Jian Jiao, Manik Varma, and Gaurav Sinha. Scaling the vocabulary of non-autoregressive models for efficient generative retrieval, 2024. URL <https://arxiv.org/abs/2406.06739>.
- [45] Xinpeng Zhao, Zhaochun Ren, Yukun Zhao, Zhenyang Li, Mengqi Zhang, Jun Feng, Ran Chen, Ying Zhou, Zhumin Chen, Shuaiqiang Wang, Dawei Yin, and Xin Xin. Diffugr: Generative document retrieval with diffusion language models, 2026. URL <https://arxiv.org/abs/2511.08150>.
- [46] P. Ferragina and G. Manzini. Opportunistic data structures with applications. In *Proceedings 41st Annual Symposium on Foundations of Computer Science*, pages 390–398, 2000. doi: 10.1109/SFCS.2000.892127.
- [47] Peiwen Yuan, Xinglin Wang, Shaoxiong Feng, Boyuan Pan, Yiwei Li, Heda Wang, Xupeng Miao, and Kan Li. Generative dense retrieval:

- Memory can be a burden, 2024. URL <https://arxiv.org/abs/2401.10487>.
- [48] Shanbao Qiao, Xuebing Liu, and Seung-Hoon Na. DiffusionRet: Diffusion-enhanced generative retriever using constrained decoding. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9515–9529, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.638. URL <https://aclanthology.org/2023.findings-emnlp.638/>.
- [49] Yujia Zhou, Zhicheng Dou, and Ji-Rong Wen. Enhancing generative retrieval with reinforcement learning from relevance feedback. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12481–12490, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.768. URL <https://aclanthology.org/2023.emnlp-main.768/>.
- [50] Tianchi Yang, Minghui Song, Zihan Zhang, Haizhen Huang, Weiwei Deng, Feng Sun, and Qi Zhang. Auto search indexer for end-to-end document retrieval, 2023. URL <https://arxiv.org/abs/2310.12455>.
- [51] Qiaoyu Tang, Jiawei Chen, Zhuoqun Li, Bowen Yu, Yaojie Lu, Cheng Fu, Haiyang Yu, Hongyu Lin, Fei Huang, Ben He, Xianpei Han, Le Sun, and Yongbin Li. Self-retrieval: End-to-end information retrieval with one large language model, 2024. URL <https://arxiv.org/abs/2403.00801>.
- [52] Yidan Zhang, Ting Zhang, Dong Chen, Yujing Wang, Qi Chen, Xing Xie, Hao Sun, Weiwei Deng, Qi Zhang, Fan Yang, Mao Yang, Qingmin Liao, Jingdong Wang, and Baining Guo. Irgen: Generative modeling for image retrieval, 2024. URL <https://arxiv.org/abs/2303.10126>.
- [53] Michael Burrows and David J. Wheeler. A block-sorting lossless data compression algorithm. Technical Report 124, Digital Equipment Corporation, 1994. URL <https://web.archive.org/web/20030105080431/https://www.hpl.hp.com/techreports/Compaq-DEC/SRC-RR-124.html>. Archived copy of the original report.
- [54] Fuwei Zhang, Xiaoyu Liu, Xinyu Jia, Yingfei Zhang, Shuai Zhang, Xiang Li, Fuzhen Zhuang, Wei Lin, and Zhao Zhang. Multi-level relevance document identifier learning for generative retrieval. In Wanxiang

- Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10066–10080, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.497. URL <https://aclanthology.org/2025.acl-long.497/>.
- [55] Yubao Tang, Ruqing Zhang, Jiafeng Guo, Jiangui Chen, Zuowei Zhu, Shuaiqiang Wang, Dawei Yin, and Xueqi Cheng. Semantic-enhanced differentiable search index inspired by learning strategies, 2023. URL <https://arxiv.org/abs/2305.15115>.
 - [56] Jiehan Cheng, Zhicheng Dou, Yutao Zhu, and Xiaoxi Li. Descriptive and discriminative document identifiers for generative retrieval. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(11):11518–11526, Apr. 2025. doi: 10.1609/aaai.v39i11.33253. URL <https://ojs.aaai.org/index.php/AAAI/article/view/33253>.
 - [57] Christian Herold, Michael Kozielski, Nicholas Santavas, Yannick Versley, and Shahram Khadivi. Vocabulary customization for efficient domain-specific llm deployment, 2025. URL <https://arxiv.org/abs/2509.26124>.
 - [58] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013. URL <https://arxiv.org/abs/1308.3432>.
 - [59] Zihan Wang, Yujia Zhou, Yiteng Tu, and Zhicheng Dou. Novo: Learnable and interpretable document identifiers for model-based ir. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, CIKM '23*, page 2656–2665, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701245. doi: 10.1145/3583780.3614993. URL <https://doi.org/10.1145/3583780.3614993>.
 - [60] Yu-An Liu, Ruqing Zhang, Jiafeng Guo, Changjiang Zhou, Maarten de Rijke, and Xueqi Cheng. On the robustness of generative information retrieval models, 2024. URL <https://arxiv.org/abs/2412.18768>.
 - [61] Varsha Kishore, Chao Wan, Justin Lovelace, Yoav Artzi, and Kilian Q. Weinberger. Incdsi: Incrementally updatable document retrieval, 2024. URL <https://arxiv.org/abs/2307.10323>.

- [62] Jiafeng Guo, Changjiang Zhou, Ruqing Zhang, Jiangui Chen, Maarten de Rijke, Yixing Fan, and Xueqi Cheng. Corpusbrain++: A continual generative pre-training framework for knowledge-intensive language tasks. *ACM Trans. Inf. Syst.*, 44(1), October 2025. ISSN 1046-8188. doi: 10.1145/3763233. URL <https://doi.org/10.1145/3763233>.
- [63] Zhen Zhang, Xinyu Ma, Weiwei Sun, Pengjie Ren, Zhumin Chen, Shuaiqiang Wang, Dawei Yin, Maarten de Rijke, and Zhaochun Ren. Replication and exploration of generative retrieval over dynamic corpora, 2025. URL <https://arxiv.org/abs/2504.17519>.
- [64] Sanket Vaibhav Mehta, Jai Gupta, Yi Tay, Mostafa Dehghani, Vinh Q. Tran, Jinfeng Rao, Marc Najork, Emma Strubell, and Donald Metzler. Dsi++: Updating transformer memory with new documents, 2023. URL <https://arxiv.org/abs/2212.09744>.
- [65] Ye Wang, Xinrun Xu, and Zhiming Ding. Mindref: Mimicking human memory for hierarchical reference retrieval with fine-grained location awareness, 2025. URL <https://arxiv.org/abs/2402.17010>.
- [66] Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning Books and Movies: Towards Story-Like Visual Explanations by Watching Movies and Reading Books . In *2015 IEEE International Conference on Computer Vision (ICCV)*, pages 19–27, Los Alamitos, CA, USA, December 2015. IEEE Computer Society. doi: 10.1109/ICCV.2015.11. URL <https://doi.ieeecomputersociety.org/10.1109/ICCV.2015.11>.
- [67] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019. URL <https://arxiv.org/abs/1907.11692>.
- [68] Tarek Naous, Michael J. Ryan, Alan Ritter, and Wei Xu. Having beer after prayer? measuring cultural bias in large language models, 2024. URL <https://arxiv.org/abs/2305.14456>.
- [69] Md Abdul Aowal, Maliha T Islam, Priyanka Mary Mammen, and Sandesh Shetty. Detecting natural language biases with prompt-based learning, 2023. URL <https://arxiv.org/abs/2309.05227>.
- [70] Roberto Navigli, Simone Conia, and Björn Ross. Biases in large language models: Origins, inventory, and discussion. *J. Data and Information*

Quality, 15(2), June 2023. ISSN 1936-1955. doi: 10.1145/3597307. URL <https://doi.org/10.1145/3597307>.

- [71] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Regina Barzilay and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147/>.
- [72] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. Kilt: a benchmark for knowledge intensive language tasks, 2021. URL <https://arxiv.org/abs/2009.02252>.
- [73] Isabel O. Gallegos, Ryan A. Rossi, Joe Barrow, Md Mehrab Tanjim, Sungchul Kim, Franck Dernoncourt, Tong Yu, Ruiyi Zhang, and Nisreen K. Ahmed. Bias and fairness in large language models: A survey. *Computational Linguistics*, 50(3):1097–1179, September 2024. doi: 10.1162/coli_a_00524. URL <https://aclanthology.org/2024.cl-3.8/>.
- [74] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning, 2022. URL <https://arxiv.org/abs/2112.09118>.
- [75] Anja Reusch and Yonatan Belinkov. Reverse-engineering the retrieval process in genir models. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR ’25, page 668–677, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400715921. doi: 10.1145/3726302.3730076. URL <https://doi.org/10.1145/3726302.3730076>.
- [76] Felix Stahlberg and Bill Byrne. On nmt search errors and model errors: Cat got your tongue?, 2019. URL <https://arxiv.org/abs/1908.10090>.
- [77] R. Zhou and Eric A. Hansen. Beam-stack search: Integrating backtracking with beam search. In *International Conference on Automated*

- Planning and Scheduling*, 2005. URL <https://api.semanticscholar.org/CorpusID:11314454>.
- [78] Hang Zhang, Yeyun Gong, Yelong Shen, Jiancheng Lv, Nan Duan, and Weizhu Chen. Adversarial retriever-ranker for dense text retrieval, 2022. URL <https://arxiv.org/abs/2110.03611>.
 - [79] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023. URL <https://arxiv.org/abs/2307.08621>.
 - [80] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. URL <https://arxiv.org/abs/2312.00752>.
 - [81] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM*, 13(7):422–426, July 1970. ISSN 0001-0782. doi: 10.1145/362686.362692. URL <https://doi.org/10.1145/362686.362692>.
 - [82] Bin Fan, Dave G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. Cuckoo filter: Practically better than bloom. In *Proceedings of the 10th ACM International on Conference on Emerging Networking Experiments and Technologies*, CoNEXT ’14, page 75–88, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450332798. doi: 10.1145/2674005.2674994. URL <https://doi.org/10.1145/2674005.2674994>.
 - [83] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, January 2025. ISSN 1558-2868. doi: 10.1145/3703155. URL <http://dx.doi.org/10.1145/3703155>.
 - [84] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL <https://arxiv.org/abs/2201.11903>.
 - [85] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection, 2023. URL <https://arxiv.org/abs/2310.11511>.

- [86] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021. URL <https://arxiv.org/abs/2005.11401>.
- [87] Garima Agrawal, Tharindu Kumarage, Zeyad Alghamdi, and Huan Liu. Mindful-rag: A study of points of failure in retrieval augmented generation, 2024. URL <https://arxiv.org/abs/2407.12216>.
- [88] Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. Seven failure points when engineering a retrieval augmented generation system, 2024. URL <https://arxiv.org/abs/2401.05856>.
- [89] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10862–10878, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.585. URL <https://aclanthology.org/2024.acl-long.585/>.
- [90] Andrew Slavin Ross, Nina Chen, Elisa Zhao Hang, Elena L. Glassman, and Finale Doshi-Velez. Evaluating the interpretability of generative models by interactive reconstruction, 2021. URL <https://arxiv.org/abs/2102.01264>.
- [91] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. KILT: a benchmark for knowledge intensive language tasks. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2523–2544, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.200. URL <https://aclanthology.org/2021.naacl-main.200/>.

- [92] Yuxin Huang, Simeng Wu, Ran Song, Yan Xiang, Yantuan Xian, Shengxiang Gao, and Zhengtao Yu. Multilingual generative retrieval via cross-lingual semantic compression, 2025. URL <https://arxiv.org/abs/2510.07812>.

List of Acronyms

DGR	Distillation Enhanced Generative Retrieval (Li et al. 2024)	8
docid	document identifier	3, 4, 12, 14–18, 22, 24, 36, 37
DPR	Dense Passage Retrieval	18, 20
DR	Dense Retrieval	1, 4, 13
GR	Generative Retrieval	1–4, 8, 11–16, 18–20, 22, 24, 27, 35–37
GT	Ground Truth	9, 14, 21, 29
IR	Information Retrieval	1, 15
KILT	Knowledge Intensive Language Tasks	35
LLM	Large Language Model	4, 20
LM	Language Model	1, 3, 12, 14, 15
LTRGR	Learning to Rank in Generative Retrieval (Li et al. 2023)	8
MINDER	Multiview Identifiers Enhanced Generative Retrieval (Li et al. 2024)	2, 7–10, 15, 19, 22–37
MSMARCO	Microsoft Machine Reading Comprehension (Bajaj et al. 2018)	2, 8, 13, 18, 22–25, 27, 28
NQ	Natural Questions	2, 8, 13, 16, 22, 23, 25, 26, 28, 36
PQ	Product Quantization	14, 17
RAG	Retrieval-Augmented Generation	11, 20, 36
RLHF	Reinforcement Learning from Human Feedback	4
SEAL	Search Engines with Autoregressive LMs (Bevilacqua et al. 2022)	2, 4–10, 15, 19, 21–37
SOTA	state-of-the-art	2